

MA-SQLGrid: A Robust Multi-Agent Framework for Text-to-SQL in Power Grid Databases

Liu Bijing¹², Sun Chenglong¹², Yang Yong^{12*}

¹NARI Group Corporation (State Grid Electric Power Research Institute), Nanjing 211106, Jiangsu Province, China;

² Beijing Kedong Electric Power Control System Co., Ltd., Beijing 100080, China

*Email: yangyong1@sgepri.sgcc.com.cn

Abstract

Accessing the vast and complex databases that underpin modern power grid operations presents a significant challenge, often requiring specialized expertise in query languages like SQL. While Large Language Models (LLMs) have shown promise in translating natural language to SQL, their application in high-stakes industrial environments is hindered by issues of reliability, schema awareness, and robustness. This paper introduces MA-SQLGrid, a novel multi-agent framework designed to address these challenges by decomposing the text-to-SQL task into a collaborative workflow among specialized LLM-based agents. The framework features five distinct agents—an NLU Analyst, a Schema Cartographer, an SQL Synthesizer, a Validation Engine, and a Counterfactual Reasoner—that interact through structured protocols of negotiation, voting, and iterative feedback to ensure query accuracy and resilience. A key innovation is the explicit integration of a Counterfactual Reasoner to evaluate and enhance the causal robustness of the generated SQL, ensuring that queries are insensitive to semantically irrelevant perturbations in user input. By transforming the query generation process into an auditable, multi-agent dialogue, MA-SQLGrid not only improves performance but also provides a more trustworthy and explainable solution for natural language data access in critical engineering domains. The proposed framework establishes a new paradigm for building reliable AI systems for industrial applications where the cost of error is high.

1. Introduction

1.1 The Data Access Challenge in Modern Power Grids

The contemporary electrical power grid, often referred to as the "smart grid," represents a paradigm shift from traditional, centralized power systems to highly distributed, data-intensive cyber-physical systems. These grids generate an unprecedented volume of heterogeneous data from a wide array of sources, including supervisory control and data acquisition (SCADA) systems, smart meters, phasor measurement units (PMUs), and asset management databases. This data, frequently characterized by its time-series and columnar nature, is the lifeblood of modern grid operations, essential for predictive maintenance, real-time fault diagnosis, load balancing, and overall operational efficiency.

The effective management and interrogation of this data deluge necessitate the use of sophisticated database management systems (DBMS) engineered for exceptional reliability, performance, and security. The mission-critical nature of power generation and transmission demands systems that are fault-tolerant, offer multi-layered security for data both at rest and in transit, and are capable of distributed query processing to handle the geographically scattered nature of grid assets. These databases often feature hybrid architectures, blending in-memory storage for microsecond-level responsiveness with persistent on-disk storage for data integrity and long-term analysis. However, a persistent and significant operational bottleneck remains: accessing this vital information is largely confined to a small cadre of human experts proficient in Structured Query Language (SQL). This "query barrier" impedes the ability of grid operators, maintenance planners, and system analysts to make timely, data-driven decisions, thereby creating a critical need to democratize data access across the organization.

1.2 Limitations of Existing Text-to-SQL Systems in Industrial Contexts

Text-to-SQL technology has emerged as a promising solution to this data access problem, aiming to translate natural language questions posed by users into executable SQL queries [Guo et al., 2019; Liu et al., 2023]. The recent advent of Large Language Models (LLMs) has dramatically advanced the capabilities of these systems, enabling them to handle increasingly complex linguistic structures. Despite this progress, the direct application of state-of-the-art text-to-SQL systems in demanding industrial contexts like power grid management reveals critical deficiencies. These systems often exhibit a profound lack of schema awareness when confronted

with real-world enterprise databases, which can contain hundreds or thousands of intricately linked tables and columns [Li et al., 2024a; Liu et al., 2023]. This leads to common failure modes such as "hallucinating" non-existent column names, misunderstanding nuanced domain-specific terminology, and generating non-optimized queries that can strain database resources [Li et al., 2024a].

These failures stem from a fundamental conflict between the nature of the technology and the requirements of the domain. Current LLMs are powerful probabilistic models, but their outputs are not inherently deterministic or guaranteed to be correct. This probabilistic nature is fundamentally at odds with the high-reliability, high-stakes environment of a power grid, where an inaccurate query result is not merely an inconvenience but a potential operational risk. A misleading result from a query about asset maintenance schedules could lead to misallocated resources, while an incorrect fault analysis could delay critical repairs. This gap between the capabilities of current LLMs and the stringent requirements of industrial applications constitutes a crisis of trust, necessitating a new architectural approach that can enforce reliability and determinism upon a probabilistic foundation.

1.3 A Multi-Agent Approach for Robust and Schema-Aware Querying

To bridge this critical gap, this paper proposes a paradigm shift away from monolithic, single-model approaches toward a collaborative, multi-agent system (MAS). MAS architectures, which comprise multiple autonomous agents interacting to solve a common problem, are inherently well-suited for complex, distributed challenges. Their modularity allows for specialization, their decentralized nature enhances robustness, and their scalability enables them to tackle problems beyond the capacity of a single agent.

We introduce **MA-SQLGrid**, a novel framework that leverages these principles to create a trustworthy text-to-SQL system for power grid databases. In MA-SQLGrid, the complex task of translating a natural language question into a validated SQL query is decomposed and assigned to a team of five specialized LLM-based agents. Each agent assumes a distinct role: one agent deconstructs the user's linguistic intent, another maps this intent to the complex database schema, a third synthesizes the SQL code, a fourth validates its execution, and a fifth assesses its causal robustness. These agents collaborate through structured interaction protocols, engaging in negotiation to resolve ambiguity and employing iterative feedback loops to correct errors. This collaborative process mimics a team of human experts working together, building a "scaffolding of trust" around the core LLM-driven generation process to ensure the final output is accurate, reliable, and robust.

1.4 Contributions and Paper Organization

The primary contributions of this work are threefold, addressing key weaknesses in current text-to-SQL systems when applied to critical infrastructure:

1. **A Novel Multi-Agent Framework (MA-SQLGrid):** We design and formalize a multi-agent architecture with five specialized roles (NLU Analyst, Schema Cartographer, SQL Synthesizer, Validation Engine, Counterfactual Reasoner). This division of labor is specifically tailored to the challenges of text-to-SQL in complex industrial databases, separating the concerns of linguistic understanding, schema mapping, code generation, and validation.
2. **Robust Interaction Protocols:** We define and implement structured protocols for negotiation, voting, and iterative feedback. These protocols govern agent collaboration, enabling the system to collectively resolve semantic and schematic ambiguities, systematically correct execution errors, and converge on a high-quality query.
3. **Causal and Counterfactual Robustness:** We introduce a novel and crucial component to the text-to-SQL pipeline: a dedicated agent for causal reasoning. This agent evaluates and improves the query's robustness against semantic perturbations in the user's question, ensuring the system's output is based on the underlying causal intent rather than superficial linguistic cues—a critical and often overlooked requirement for trustworthy AI.

The remainder of this paper is organized as follows. Section 2 provides a comprehensive review of related work in text-to-SQL, multi-agent systems, and data management in engineering domains. Section 3 presents a formal problem definition and an overview of the MA-SQLGrid framework. Section 4 offers a detailed exposition of the agent roles and interaction protocols. Section 5 describes the experimental setup, including the design of a simulated power grid database for evaluation. Section 6 presents the experimental results and a thorough analysis, including ablation studies. Finally, Section 7 concludes the paper with a summary of findings and directions for future research.

2. Related Work

2.1 The Evolution of Text-to-SQL: From Sequential Models to

Schema-Aware Transformers

The field of text-to-SQL has undergone a significant evolution, moving from simplistic rule-based systems to sophisticated neural architectures. Early attempts relied on manually crafted rules and heuristics, which were deterministic but proved to be brittle, difficult to scale, and unable to handle linguistic variation or complex database schemas [Liu et al., 2023].

The application of deep learning marked a major turning point. A seminal work in this area was **Seq2SQL** [Zhong et al., 2017], which framed the problem as a sequence-to-sequence translation task. Seq2SQL introduced a model based on Long Short-Term Memory (LSTM) networks and notably employed policy-based reinforcement learning to generate the WHERE clause of a query. This innovative use of RL allowed the model to handle the inherently unordered nature of SQL conditions, optimizing directly for execution accuracy rather than exact string matching [Zhong et al., 2017]. The paper also introduced the **WikiSQL** dataset, a large-scale corpus of single-table questions that became a standard benchmark for subsequent research [Zhong et al., 2017].

While powerful, early neural models like Seq2SQL struggled with a critical challenge: generalization to unseen database schemas. They implicitly learned schema information from the training data, making them ineffective in cross-domain settings where the test set contained new databases. This limitation spurred the development of schema-aware models. Among the most influential of these was **RAT-SQL**. RAT-SQL introduced a unified framework based on a relation-aware self-attention mechanism, an extension of the Transformer architecture. It explicitly encodes the database schema as a graph, with tables and columns as nodes and relationships (e.g., primary-foreign keys) as edges. The model then jointly reasons over the natural language question and this schema graph, enabling it to effectively model the alignment between linguistic mentions and database elements. This approach set a new state-of-the-art on the **Spider** dataset, a challenging benchmark designed specifically for complex, cross-domain text-to-SQL tasks.

2.2 Large Language Models for Code Generation and Semantic Parsing

The emergence of large-scale, pre-trained language models (LLMs) such as the Generative Pre-trained Transformer (GPT) family has catalyzed another paradigm shift in text-to-SQL and code generation more broadly. LLMs demonstrate a remarkable capacity to understand and generate programming code, including SQL, from natural language prompts, often with minimal task-

specific training (i.e., in zero-shot or few-shot settings) [Chen et al., 2021; Li et al., 2024a]. This has enabled rapid development of powerful text-to-SQL systems.

However, the application of general-purpose LLMs to this task is not without significant challenges. Their performance is highly sensitive to the structure and content of the input prompt, a discipline known as prompt engineering. They struggle with complex database schemas that exceed their fixed context window limits, forcing developers to devise strategies for schema pruning and serialization. Most critically, LLMs are prone to generating queries that are syntactically invalid (i.e., will not parse) or semantically incorrect (i.e., will execute but return the wrong answer), a phenomenon often linked to "hallucinations" [Li et al., 2024a].

Several state-of-the-art methods have been developed to mitigate these weaknesses. The **Picard** framework addresses the issue of syntactic validity by introducing a method for constrained auto-regressive decoding. It integrates an incremental parser into the beam search decoding process, effectively pruning any token sequence that violates the formal grammar of SQL. This guarantees that the final output is always a syntactically valid query without requiring any modification to the underlying pre-trained model. Other approaches, such as **GPT-SQL**, focus on enhancing semantic correctness through sophisticated prompt construction. These methods enrich the prompt with detailed schema knowledge, domain-specific context (such as geographic information for spatial databases), and a self-correction loop where the model refines its query based on feedback from the database execution engine. Similarly, the **DAIL-SQL** solution demonstrates that carefully selecting and organizing few-shot examples based on query skeleton similarity can significantly improve both accuracy and token efficiency, which is a critical economic consideration when using large commercial models.

This progression reveals an ongoing effort to provide models with richer, more relevant context. Seq2SQL contextualized words within a sequence; RAT-SQL contextualized the question within a structural schema graph; and LLMs contextualize the query within a vast repository of world knowledge. The inherent weakness of the current LLM paradigm is that this world knowledge is often too general and not sufficiently anchored to the rigid, specific rules of a given database schema. This creates a "context gap" that leads to errors. The MA-SQLGrid framework proposes the next step in this evolution: **social contextualization**. The query is interpreted not by a single model, but through a structured dialogue among specialized agents. This collaborative process creates a dynamic, task-specific context that is richer, more grounded, and self-correcting than any static context provided in a single prompt to a monolithic LLM.

2.3 Collaborative AI: Multi-Agent Systems in Natural Language Processing

A Multi-Agent System (MAS) is a computational system composed of multiple autonomous, interacting agents that collaborate to solve problems that are difficult or impossible for a single agent to solve. The core abilities that distinguish LLM-powered agents within a MAS are their capacity for adaptability (learning from interaction), reasoning (planning and problem decomposition), and cooperation (task allocation and communication). The architecture of these systems can vary, with common patterns including centralized or supervisor-based models, decentralized or networked models, and more complex hierarchical structures. The MA-SQLGrid framework adopts a supervisor-based architecture, where a central Orchestrator agent manages the workflow and delegates tasks to a team of specialized agents.

The application of LLM-based MAS to complex Natural Language Processing (NLP) tasks is a rapidly emerging field of research. Studies have shown that distributing a complex task among a team of specialized agents can significantly enhance performance in areas such as long-term planning, generalization to new problems, and overall efficiency. This approach is inspired by the principles of human teamwork and the division of labor, where individual experts combine their unique skills and perspectives to achieve a collective goal that surpasses the sum of their individual capabilities. By assigning specific roles and tools to each agent, the system can tackle multifaceted problems in a more structured and robust manner than a single, monolithic model.

2.4 Data Interrogation in Critical Engineering Domains

Applying NLP and database technologies to critical engineering domains, such as power systems, introduces a unique set of challenges that are not typically present in general-purpose applications. The data in these domains is often highly distributed, reflecting the physical layout of infrastructure. Database schemas are exceptionally complex, laden with domain-specific terminology and intricate relationships that require expert knowledge to navigate. Most importantly, the cost of an error is exceptionally high. An incorrect query result in a business analytics dashboard might lead to a flawed marketing decision; in a power grid control system, it could have implications for system stability and safety.

Consequently, prior research in this area has heavily focused on the foundational layers of the data stack. This includes the development of DBMS specifically designed for mission-critical reliability, real-time performance, and fault tolerance. It also includes the application of data mining and machine learning techniques for specific analytical tasks like fault classification, power quality analysis, and predictive load forecasting. However, the development of a natural language interface for these complex, high-stakes systems remains a largely underexplored area of research. MA-SQLGrid aims to fill this gap by proposing an architecture designed from the ground up to meet the stringent requirements of reliability, accuracy, and robustness demanded

by such critical engineering domains.

3. Problem Formulation and Framework Overview

3.1 Mathematical Formulation of the Text-to-SQL Task

The text-to-SQL task can be formally defined as the process of learning a mapping function, denoted as f , that translates a user's request from natural language into a structured, executable SQL query. This mapping is conditioned on the specific structure of the target database.

The function is expressed as:

$$f: (Q, S) \rightarrow Y$$

The inputs to this function are a pair (Q, S) :

- **Natural Language Question (Q):** This is the user's query, represented as a sequence of tokens $Q = (q_1, q_2, \dots, q_m)$, where each q_i is a word or sub-word token from the user's utterance.
- **Database Schema (S):** This is a formal, structured representation of the database against which the query will be executed. The schema S defines the set of tables $T = \{t_1, t_2, \dots, t_N\}$, where each table t_i has a corresponding set of columns $C_i = \{c_{i,1}, c_{i,2}, \dots, c_{i,k_i}\}$. Crucially, the schema also defines the relationships between tables, typically through primary key and foreign key constraints, which are essential for constructing correct JOIN operations.

The output of the function is Y :

- **SQL Query (Y):** This is the target executable SQL query, represented as a sequence of tokens $Y = (y_1, y_2, \dots, y_n)$. The vocabulary for these tokens is a combination of standard SQL keywords (e.g., SELECT, FROM, WHERE, GROUP BY), aggregation operators (COUNT, SUM, AVG), and the specific names of tables and columns drawn from the input schema S .

3.2 Defining Optimization Objectives

While traditional text-to-SQL research has primarily focused on maximizing execution accuracy, deploying such systems in critical industrial environments necessitates a more comprehensive set of optimization objectives. The MA-SQLGrid framework is designed to optimize for a hierarchy of objectives that collectively ensure the trustworthiness and reliability of the generated queries.

1. **Schema Alignment:** This is the foundational objective. The generated query Y must be perfectly aligned with the given schema S . This means that every table and column referenced in Y must exist in S , all join paths must be valid according to the defined foreign key relationships, and operations must respect the data types of the columns involved (e.g., performing arithmetic operations only on numeric types) [Liu et al., 2023]. A failure in schema alignment results in a query that is syntactically invalid or will cause an execution error.
2. **Semantic Coverage:** Building upon a correctly aligned query, this objective requires that Y accurately and completely captures the full semantic intent of the user's question Q . This involves correctly identifying all entities (e.g., "transformers," "substation A"), predicates (e.g., "installed before 2020"), and constraints (e.g., "show the top 5") mentioned in the natural language and translating them into the appropriate SQL clauses (WHERE, ORDER BY, LIMIT, etc.) [Liu et al., 2023]. A failure in semantic coverage leads to a query that executes but returns an incorrect or incomplete answer.
3. **Causal and Counterfactual Robustness:** This is the highest-level objective, ensuring the reliability of the model's reasoning. The mapping function f should be robust to perturbations in the input question Q that do not alter its core causal intent. For example, if a user asks, "Which transformers have a high failure rate?" the system should produce the same semantic query as for the counterfactual question, "Show me the transformers with a large number of faults." A robust system should not be swayed by the specific choice of words ("high failure rate" vs. "large number of faults") but should instead recognize the underlying causal link between faults and failure rate. This ensures the model is learning genuine semantic relationships rather than relying on spurious correlations in the training data, a critical property for trustworthy AI [Chang et al., 2023].

The hierarchical nature of these objectives—where schema alignment is a prerequisite for semantic coverage, and both are prerequisites for robustness—directly informs the pipelined, multi-stage workflow of the MA-SQLGrid architecture. Each stage of the agent collaboration is designed to satisfy one of these objectives before passing control to the next, ensuring a structured and rigorous query construction process.

3.3 High-Level Architecture of the MA-SQLGrid System

To achieve these hierarchical objectives, MA-SQLGrid is architected as a supervised, hierarchical multi-agent system. This design choice allows for a clear separation of concerns and a structured, auditable workflow.

At the heart of the system is an **Orchestrator Agent**, which functions as the supervisor. It receives the initial user query and schema, and then manages the entire query generation lifecycle. It does not perform the core NLP or SQL generation tasks itself; instead, it delegates these sub-tasks to a team of specialized agents. The Orchestrator is responsible for invoking the correct agent at each stage of the process, passing the necessary information between them, and making the final decision on when the process is complete.

The specialized agents—the NLU Analyst, Schema Cartographer, SQL Synthesizer, Validation Engine, and Counterfactual Reasoner—each have a narrowly defined role. They receive inputs from the Orchestrator, perform their specific function, and report their findings back. This interaction is not a simple linear sequence but a dynamic, collaborative process. For instance, if the Validation Engine detects an error, the Orchestrator initiates a feedback loop, sending the error message back to the SQL Synthesizer for correction. This entire workflow is modeled as a state graph, where nodes represent agent actions and edges represent the flow of control and information, a concept similar to that used in modern agentic frameworks like LangGraph. This architecture ensures that the complex problem of generating a robust SQL query is broken down into manageable, verifiable steps.

4. The MA-SQLGrid Framework: Architecture and Protocols

The MA-SQLGrid framework is predicated on the principles of division of labor and collaborative problem-solving. Its architecture is defined by the specialized roles of its constituent agents and the structured protocols that govern their interactions. This design transforms the opaque, internal reasoning of a single LLM into an explicit, auditable dialogue between experts, fundamentally enhancing the system's transparency and trustworthiness.

4.1 Agent Specialization and Role Design

Each agent within the MA-SQLGrid is an instance of a powerful LLM, but it is not a general-purpose entity. Instead, each is instantiated with a carefully crafted system prompt that assigns it a specific role, a clear goal, and a limited set of tools it is permitted to use. This technique of "role-playing" focuses the LLM's capabilities on a single sub-task, leading to higher performance and more predictable behavior compared to a monolithic approach.

4.1.1 The NLU Analyst Agent

- **Role:** To serve as the system's natural language understanding expert, responsible for the initial deconstruction of the user's query, Q.
- **Tasks:** The NLU Analyst performs a series of classical NLP tasks to translate the unstructured user input into a structured format. This includes:
 - **Intent Recognition:** Identifying the user's primary goal, such as performing an aggregation (COUNT, AVG), retrieving a list of entities, or finding an extreme value (MAX, MIN) [Liu et al., 2023].
 - **Entity Extraction:** Pinpointing key pieces of information within the query, such as specific asset identifiers ("transformer T-101"), locations ("downtown substation"), temporal constraints ("last six months"), and value-based conditions ("voltage spikes above 5%") [Liu et al., 2023].
 - **Ambiguity Identification:** Proactively flagging terms that are likely to be ambiguous in the context of the database. For example, a query for "status" could refer to the operational status of an asset or the status of a work order. It also recognizes enterprise-specific jargon, such as "pipeline," which might implicitly mean "current quarter" in a business context.
- **Output:** The agent produces a structured data object (e.g., a JSON object) that contains the identified intent, a list of extracted entities with their types (e.g., temporal, location, asset_id), and a list of any detected ambiguities that require resolution in the next stage.

4.1.2 The Schema Cartographer Agent

- **Role:** To act as the bridge between the linguistic domain of the user's question and the rigid, structural domain of the database schema, S. Its primary function is to address the "schema alignment" objective.
- **Tasks:**
 - **Schema Linking:** The agent takes the structured output from the NLU Analyst and maps the extracted entities to the most probable tables and columns in the database

schema [Liu et al., 2023].

- **Schema Pruning:** Power grid databases can be enormous. To manage the LLM's context window limitations and reduce the search space for the SQL generation step, the Cartographer identifies a minimal, relevant sub-schema containing only the tables and columns necessary to answer the query.
- **Join Path Identification:** Using the foreign key relationships defined in the schema, the agent traverses the schema graph to identify all valid join paths between the tables in the relevant sub-schema. This prevents the generation of queries with incorrect or illogical joins [Liu et al., 2023].
- **Output:** A pruned schema containing only relevant tables and columns, and a ranked list of potential join paths, ordered by simplicity (e.g., fewest joins).

4.1.3 The SQL Synthesizer Agent

- **Role:** To function as the system's expert SQL programmer, responsible for generating one or more candidate SQL queries.
- **Tasks:** The Synthesizer receives the structured query components from the NLU Analyst and the pruned, relevant sub-schema from the Schema Cartographer. Its task is to synthesize these inputs into a coherent SQL query. For complex queries involving nested subqueries or multiple GROUP BY clauses, the agent can be prompted to use a Chain-of-Thought process, breaking down the problem into smaller logical steps before writing the final code. To guarantee syntactic correctness, the agent's token-by-token generation process can be constrained by a formal SQL grammar, ensuring that the output is always parsable.
- **Output:** A set of one or more candidate SQL queries, $\{Y1, Y2, \dots, Yk\}$, that represent plausible translations of the user's request.

4.1.4 The Validation Engine Agent

- **Role:** To act as the system's quality assurance tester, responsible for rigorously verifying the correctness and executability of the generated queries.
- **Tasks:** This agent is equipped with a tool that allows it to connect to a sandboxed, read-only replica of the power grid database. It takes each candidate query from the SQL Synthesizer and attempts to execute it. The agent checks for several types of failures:
 - **Syntax Errors:** The query violates the SQL dialect of the database.
 - **Execution Errors:** The query is syntactically valid but fails during execution (e.g., due to a type mismatch or division by zero).

- **Empty or Anomalous Results:** The query executes successfully but returns an empty result set, which can often indicate a logical error in the query (e.g., an overly restrictive WHERE clause).
- **Output:** For each candidate query, the agent returns a status report containing: an execution status (e.g., SUCCESS, SYNTAX_ERROR, EXECUTION_ERROR), the result of the query if successful, and the specific error message from the DBMS if it failed.

4.1.5 The Counterfactual Reasoner Agent

- **Role:** To serve as the system's "red team" expert, tasked with assessing and enhancing the causal robustness of the final, validated query. This is a novel role in text-to-SQL systems, designed to ensure reliability.
- **Tasks:**
 - **Counterfactual Generation:** The agent takes the original, validated user question Q and generates a set of semantically equivalent but linguistically varied perturbations. For example, it might replace keywords with synonyms ("substation" → "transformer station"), rephrase active voice to passive voice, or alter the sentence structure [Chang et al., 2023].
 - **Consistency Check:** For each generated counterfactual question Q', the agent runs it through a truncated version of the MA-SQLGrid pipeline (NLU, Schema, SQL agents) to generate a new query Y'. It then compares the abstract syntax tree or logical form of Y' with that of the original validated query Y.
 - **Robustness Scoring:** If the generated queries are semantically equivalent across the perturbations, the original query is deemed robust. If inconsistencies arise, it indicates that the model may be relying on spurious correlations in the original phrasing rather than the core causal intent of the question.
- **Output:** A numerical robustness score and, in cases of failure, feedback that can be used to refine the prompting strategies for the other agents to make them less sensitive to superficial linguistic variations.

4.2 Multi-Agent Interaction Protocols

The collaboration between these specialized agents is not unstructured. It is governed by a set of formal interaction protocols that ensure the workflow is efficient, effective, and auditable.

4.2.1 Negotiation for Schema Ambiguity Resolution

- **Trigger:** This protocol is invoked by the Orchestrator when the NLU Analyst flags an ambiguous term in the user's query, and the Schema Cartographer confirms that this term maps to multiple potential tables or columns in the schema.
- **Protocol:** A structured negotiation dialogue commences between the NLU Analyst and the Schema Cartographer. This process follows principles of automated negotiation, involving an exchange of proposals and justifications to reach a mutually agreeable resolution.
 1. **Proposal:** The Schema Cartographer proposes the most likely mapping based on schema-level heuristics, such as column popularity or frequency of use in historical queries, information that can be stored in a "context layer".
 2. **Argumentation:** The NLU Analyst evaluates this proposal against the linguistic context of the full query. It may provide a counter-argument, such as, "The user also mentioned 'technician,' which suggests the 'status' in the Work_Orders table is more relevant than the 'status' in the Assets table."
 3. **Convergence:** The agents exchange information iteratively until they converge on the mapping with the highest joint probability, which is then used for subsequent steps. This dialogue provides a clear, explainable trace of how ambiguity was resolved.

4.2.2 Voting Mechanisms for Query Candidate Selection

- **Trigger:** This protocol is used when the SQL Synthesizer generates multiple candidate queries, and the Validation Engine confirms that two or more of them execute successfully and produce non-empty results.
- **Protocol:** To select the single best query, a voting or consensus mechanism is employed. Research on multi-agent debates has shown that voting is particularly effective for reasoning tasks, which aligns with the nature of selecting the best logical representation for a query.
 1. **Candidate Presentation:** The Orchestrator presents the set of valid candidate queries to the relevant agents.
 2. **Voting:** Each agent casts a vote or a ranked preference based on its specialized expertise.
 - The **NLU Analyst** votes based on which query best captures the full semantic coverage of the original question.
 - The **Schema Cartographer** votes for the query that uses the most direct join paths

- or has the lowest structural complexity.
- The **Validation Engine** can provide a preliminary performance score (e.g., query execution time) as its vote.
3. **Tallying:** The Orchestrator tallies the votes and selects the query with the highest score to proceed to the final robustness check.

4.2.3 Iterative Feedback Loops for Self-Correction and Refinement

- **Trigger:** This is one of the most critical protocols, initiated whenever the Validation Engine reports a query execution error.
- **Protocol:** The system enters a self-correction loop, a process inspired by the iterative way human data analysts debug their own SQL code.
 1. **Feedback Propagation:** The Orchestrator captures the detailed error message returned by the database (e.g., "no such column: asset_name") and the failed SQL query.
 2. **Refinement Prompt:** This feedback is sent back to the SQL Synthesizer. The prompt is framed as a debugging task: "The following query failed with the error: [error_message]. Based on the original question and schema, please correct the query."
 3. **Re-generation:** The SQL Synthesizer generates a new, corrected version of the query.
 4. **Re-validation:** The corrected query is sent back to the Validation Engine for another execution attempt.
 5. **Termination:** This loop continues until a query executes successfully or a predefined maximum number of retries (e.g., 3) is reached, at which point the system reports a failure to the user. This prevents infinite loops and ensures timely feedback.

5. Experimental Setup

To rigorously evaluate the performance, domain adaptability, and robustness of the MA-SQLGrid framework, a comprehensive experimental plan was designed. This plan involves testing on both widely recognized public benchmarks and a novel, domain-specific simulated database, and comparing the results against a range of competitive baseline models using a multi-faceted set of evaluation metrics.

5.1 Datasets

The choice of datasets is critical for assessing the different facets of the framework's capabilities. The evaluation leverages both general-purpose benchmarks to measure cross-domain generalization and a specialized dataset to test performance in the target industrial context.

5.1.1 Public Benchmarks

To ensure comparability with the broader text-to-SQL literature, two standard public datasets are used:

- **Spider:** This is a large-scale, complex, and cross-domain dataset that has become the de facto standard for evaluating a model's ability to generalize to unseen database schemas. It consists of 10,181 questions and 5,693 unique SQL queries across 200 databases covering 138 different domains. The databases feature multiple tables with foreign key relationships, and the queries often involve complex clauses like JOIN, GROUP BY, and nested subqueries. The cross-domain nature of its training/testing split makes it an ideal benchmark for assessing schema-aware reasoning.
- **WikiSQL:** This dataset is one of the largest text-to-SQL corpora, containing 80,654 hand-annotated examples over 24,241 tables from Wikipedia [Zhong et al., 2017]. Unlike Spider, the queries in WikiSQL are simpler and target only a single table at a time. It is useful for evaluating the model's performance on more straightforward, direct questions and serves as a valuable point of comparison for foundational text-to-SQL capabilities [Zhong et al., 2017].

5.1.2 Simulated Power Grid Maintenance Database (GridDB-Maintenance)

A significant limitation of existing benchmarks is their lack of relevance to specific, high-stakes industrial domains. To address this, a realistic, synthetic database named **GridDB-Maintenance** was created to serve as a domain-specific evaluation testbed.

- **Motivation:** No publicly available dataset adequately captures the schematic complexity, domain-specific terminology, and typical query patterns associated with power grid asset management and maintenance operations. This simulated database provides a challenging and realistic environment to test MA-SQLGrid's efficacy in its target application area.
- **Schema Design:** The database schema was meticulously designed based on principles of

power grid asset management and data modeling found in engineering literature and industry best practices. The relational schema, implemented in PostgreSQL, includes the following core tables and relationships:

- Assets (PK: asset_id, Attributes: asset_type_id, location_id, installation_date, operational_status)
- Asset_Types (PK: type_id, Attributes: type_name, description, expected_lifespan)
- Locations (PK: location_id, Attributes: substation_name, geo_coordinates)
- Maintenance_Logs (PK: log_id, FK: asset_id, technician_id, Attributes: maintenance_type, log_date, notes)
- Work_Orders (PK: order_id, FK: asset_id, Attributes: issue_date, completion_date, status, priority)
- Sensor_Readings (PK: reading_id, FK: asset_id, Attributes: timestamp, reading_type, value, unit)
- Grid_Topology (PK: link_id, FKs: source_asset_id, target_asset_id, Attributes: relationship_type)
- Technicians (PK: technician_id, Attributes: name, specialization)
- **Data and Query Generation:** The tables were populated with synthetically generated data representing a plausible medium-sized utility's assets over a five-year period. A corpus of 1,000 question-SQL pairs was then manually authored by domain experts. These pairs were designed to reflect realistic information needs of grid operators and maintenance planners, covering scenarios such as: "List all circuit breakers in the 'North Valley' substation that are older than 10 years and have not had preventative maintenance in the last year," and "What is the average hotspot temperature reading for transformer 'TX-451' during last month's heatwave?"

Dataset	# Databases	# Domains	# Questions	# Unique SQLs	Avg. Tables/DB	Avg. Query Tokens	Key Characteristics
Spider	200	138	10,181	5,693	5.1	21.0	Cross-domain, multi-table, complex joins, nested
WikiSQ	24,241	1	80,654	77,840	1.0	14.5	Single-domain

L							(Wikipedia), single-table, simple
GridDB - Maintenance	1	1	1,000	950	8.0	28.5	Domain-specific (Power Grid), complex joins, temporal

Table 1: Characteristics of Evaluation Datasets

5.2 Competitive Baselines

To contextualize the performance of MA-SQLGrid, it is compared against a carefully selected set of four baseline models that represent the historical evolution and current state-of-the-art in text-to-SQL:

1. **Seq2SQL** [Zhong et al., 2017]: A foundational sequence-to-sequence model with reinforcement learning. It serves as a representative of early neural approaches that are not explicitly schema-aware.
2. **RAT-SQL**: A state-of-the-art schema-aware model that uses a relation-aware Transformer. It represents the pinnacle of models developed before the widespread adoption of very large pre-trained language models.
3. **Picard + T5-3B**: A strong baseline representing modern LLM-based approaches. It combines the powerful T5-3B language model with the Picard constrained decoding algorithm, which guarantees the generation of syntactically valid SQL.
4. **GPT-4 (Zero-shot)** [Achiam et al., 2023]: This baseline uses a state-of-the-art, general-purpose LLM (GPT-4) in a zero-shot setting, with only the question and schema provided in the prompt. It represents a "naive" but powerful LLM approach, without the benefit of

specialized prompting, few-shot examples, or the collaborative framework of MA-SQLGrid.

5.3 Evaluation Metrics and Robustness Assessment

A multi-faceted evaluation protocol is employed to provide a holistic assessment of model performance, moving beyond single metrics to capture functional correctness, structural alignment, and system robustness.

5.3.1 Primary Metrics

- **Execution Accuracy (EX):** This is the primary metric for functional correctness. It is defined as the percentage of generated queries that, when executed against the database, produce a result set identical to the result set of the ground-truth query. This metric is tolerant of syntactic variations as long as the final output is correct [Zhong et al., 2017].
- **Exact Match (EM):** This is a stricter metric that measures the percentage of generated queries that are an exact string match to the ground-truth query after a canonical representation (e.g., lowercasing, removing whitespace). It penalizes functionally equivalent queries that are written differently.

5.3.2 Schema Alignment Metrics

To provide a more fine-grained analysis of how well models handle schema, especially in the context of perturbations, we adopt metrics from the EvoSchema benchmark:

- **Table Match F1 & Column Match F1:** These metrics calculate the F1 score for the set of tables and columns, respectively, that are correctly identified in the generated query compared to the ground-truth query. This helps diagnose failures related specifically to schema linking.

5.3.3 Robustness Evaluation

A critical component of the evaluation is to assess the robustness of each model to realistic variations in both the natural language questions and the database schemas. A counterfactual test set is created by applying systematic perturbations to the development sets of Spider and GridDB-Maintenance, inspired by the methodologies of robustness benchmarks like Dr. Spider and EvoSchema [Chang et al., 2023; Pi et al., 2022].

- **Perturbation Types:**
 - **Question Perturbations:** These include replacing keywords with synonyms (e.g., "show" → "list"), rephrasing sentences from active to passive voice, and adding irrelevant clauses.
 - **Schema Perturbations:** These involve modifications to the database schema provided to the model, such as renaming columns or tables with plausible synonyms (e.g., `Assets.operational_status` → `Assets.current_state`) and adding irrelevant "distractor" columns to tables.
- **Robustness Metric:** The primary metric for robustness is the **percentage drop in Execution Accuracy** on the perturbed dataset compared to the original, unperturbed dataset. A model with a smaller drop is considered more robust, as its performance is less affected by these common, real-world variations.

6. Results and Analysis

This section presents the empirical evaluation of the MA-SQLGrid framework. The analysis is structured to first establish its performance against baselines on standard benchmarks and the specialized power grid domain, then to dissect the contributions of its individual components through ablation studies, and finally to rigorously assess its primary design goal: robustness.

6.1 Performance on Standard Benchmarks

The initial set of experiments was conducted on the test sets of the Spider and WikiSQL benchmarks to evaluate the general text-to-SQL capabilities of MA-SQLGrid in comparison to the established baselines. The results, summarized in Table 2, demonstrate the effectiveness of the multi-agent collaborative approach.

On the complex, cross-domain Spider dataset, MA-SQLGrid achieves a state-of-the-art Execution Accuracy (EX) of 88.2%, significantly outperforming all baselines. Notably, it

surpasses the strong Picard + T5-3B model by 12.7 percentage points and the naive GPT-4 (Zero-shot) baseline by 15.5 points. This substantial improvement highlights the framework's advanced capability in handling complex schemas, multi-table joins, and nested queries, which are prevalent in Spider. The iterative self-correction and schema-linking negotiation protocols are particularly effective in resolving the types of errors that monolithic models frequently make on this benchmark.

On the simpler, single-table WikiSQL dataset, the performance gap narrows, as expected. However, MA-SQLGrid still leads with an EX of 93.5%. The high performance of all LLM-based models on this dataset indicates that simple, single-table queries are largely a solved problem for modern architectures, but the marginal gain by MA-SQLGrid suggests its validation and refinement steps help eliminate the "long tail" of errors even in less complex scenarios.

6.2 Efficacy in the Power Grid Domain

Performance on the custom-built GridDB-Maintenance dataset provides the most direct evidence of the framework's suitability for its target industrial application. As shown in Table 2, MA-SQLGrid achieves an EX of 91.7%, demonstrating its ability to adapt to a highly specialized and complex domain.

The baseline models, particularly those not explicitly designed for schema-awareness like Seq2SQL, struggle significantly with the domain-specific terminology and the intricate relational structure of the power grid database. Even the powerful GPT-4 model, in a zero-shot setting, achieves only 71.3% EX, indicating that general world knowledge is insufficient to navigate the nuances of a specialized engineering schema. The success of MA-SQLGrid in this context can be attributed to the Schema Cartographer's ability to effectively prune the schema and identify correct join paths, and the NLU Analyst's capacity to link domain-specific terms like "hotspot temperature" and "preventative maintenance" to their corresponding schema elements.

Model	Spider (EX %)	Spider (EM %)	WikiSQL (EX %)	WikiSQL (EM %)	GridDB- Maintena nce (EX %)	GridDB- Maintena nce (EM %)
Seq2SQL	19.8	12.4	60.3	49.2	23.4	15.8

RAT-SQL	65.6	59.8	78.9	74.6	58.1	51.3
Picard + T5-3B	75.5	71.9	91.8	89.5	69.2	65.5
GPT-4 (Zero-shot)	72.7	68.3	92.1	90.3	71.3	66.9
MA-SQLGrid (ours)	88.2	83.5	93.5	91.7	91.7	87.4

Table 2: Main Performance Results vs. Baselines (%) on Execution Accuracy (EX) and Exact Match (EM)

6.3 Ablation Studies: Quantifying Agent Contributions

To validate the multi-agent architecture and understand the contribution of each specialized agent, a series of ablation studies were conducted on the Spider development set. In each study, a single agent component was removed from the full framework, and the resulting impact on Execution Accuracy was measured. The results, presented in Table 3, confirm that each agent plays a critical and synergistic role in the system's overall performance.

Removing the **Validation Engine** causes the most significant performance degradation, with a drop of 24.6 percentage points. This result is profound, as it demonstrates the inadequacy of relying on a single-shot generation from an LLM, even a powerful one. The iterative self-correction loop, which allows the system to learn from its own mistakes using database feedback, is clearly the most impactful component for achieving high accuracy.

The removal of the **Schema Cartographer** results in the second-largest drop (18.9 points). This highlights the critical importance of schema-aware reasoning. When the full, unpruned schema is passed to the SQL Synthesizer, it becomes overwhelmed by irrelevant tables and columns,

leading to frequent errors in table selection and join path construction. This confirms that targeted schema linking and pruning are essential for applying LLMs to large, complex databases.

Disabling the **NLU Analyst** and the **Counterfactual Reasoner** leads to smaller but still highly significant drops of 7.1 and 5.8 points, respectively. The NLU Analyst's contribution comes from structuring the query and flagging ambiguities upfront, which simplifies the task for downstream agents. The Counterfactual Reasoner's impact, even on a non-perturbed dataset, suggests that the process of checking for causal robustness implicitly encourages the generation of more generalizable and logically sound queries.

The analysis reveals more than just the individual importance of each agent; it points to a non-linear, synergistic relationship between them. The performance degradation from removing two components, such as the Schema Cartographer and the Validation Engine, would likely be far greater than the sum of their individual drops. The agents are not merely additive; they form a tightly coupled system where each component multiplies the effectiveness of the others. This collaborative architecture creates an emergent intelligence and resilience that is fundamentally absent in the individual parts or in monolithic models.

Model Configuration	Execution Accuracy (EX %)	Delta (Δ) from Full Model
Full MA-SQLGrid	88.5	-
w/o Validation Engine (no self-correction)	63.9	-24.6
w/o Schema Cartographer (full schema)	69.6	-18.9
w/o NLU Analyst (raw query input)	81.4	-7.1
w/o Counterfactual Reasoner (no robustness check)	82.7	-5.8

Table 3: Ablation Study of MA-SQLGrid Components on the Spider Dev Set (EX %)

6.4 Analysis of Causal and Counterfactual Robustness

The final set of experiments was designed to test the central hypothesis that MA-SQLGrid is not only more accurate but also significantly more robust than existing models. Table 4 shows the performance of the top-performing models on the original development sets of Spider and GridDB-Maintenance versus their performance on the adversarially perturbed versions of these sets.

The results are stark. While all models experience some performance degradation, the drop is far more pronounced for the baseline models. The naive GPT-4 model, despite its high baseline accuracy, is the most fragile, suffering a 31.2% drop on the perturbed Spider set. This indicates that its reasoning is heavily reliant on superficial patterns and keyword matching in the prompt, which are easily broken by synonym replacement and rephrasing. The Picard + T5-3B model is more robust, likely due to its constrained decoding, but still sees a significant 22.5% drop.

In contrast, MA-SQLGrid demonstrates exceptional robustness, with its performance dropping by only 4.8% on the perturbed Spider set and 3.5% on the GridDB-Maintenance set. This resilience is the direct result of the Counterfactual Reasoner agent. By explicitly generating and testing against perturbations during its reasoning process, the framework learns to produce queries that are invariant to superficial linguistic changes and are grounded in the core semantic and causal intent of the user's question. This finding is the most powerful argument for the framework's suitability in real-world industrial settings, where user queries are varied and unpredictable, and reliability is paramount.

Model	Dataset	Base EX (%)	Perturbed EX (%)	% Drop
RAT-SQL	Spider Dev	66.1	51.3	22.4%
	GridDB-Maintenance	59.5	43.8	26.4%
Picard + T5-3B	Spider Dev	76.8	59.5	22.5%

	GridDB-Maintenance	70.1	55.4	20.9%
GPT-4 (Zero-shot)	Spider Dev	74.0	50.9	31.2%
	GridDB-Maintenance	72.5	53.6	26.1%
MA-SQLGrid (ours)	Spider Dev	88.5	84.2	4.8%
	GridDB-Maintenance	92.1	88.9	3.5%

Table 4: Robustness Evaluation under Question and Schema Perturbations (Execution Accuracy % and Percentage Drop)

A detailed breakdown of common error types highlights the specific advantages of the MA-SQLGrid framework's layered validation and correction mechanisms.

Error Category	Example	Baseline Failure Mode	MA-SQLGrid Mitigation
System/Syntax Errors	Invalid SQL syntax (e.g., mismatched parentheses, incorrect keywords).	Monolithic LLMs may generate syntactically flawed queries that fail to execute.	Validation Engine catches syntax errors; Iterative Feedback Loop prompts the SQL Synthesizer for correction.
Semantic Errors	Incorrect table/column	Model misunderstands user	Schema Cartographer

	selection; wrong join conditions; improper aggregation function.	intent or schema, leading to a query that runs but returns incorrect data.	ensures correct schema linking and join paths. NLU Analyst clarifies intent. Voting Mechanism selects the most semantically sound query.
Ambiguity/Unanswerable	User query maps to multiple columns (e.g., "status"); question requires data not in the schema.	Model "hallucinates" a plausible but incorrect column or provides a nonsensical answer.	NLU Analyst flags ambiguity, triggering the Negotiation Protocol with the Schema Cartographer to resolve it before query generation.

Table 5: Error Analysis by Category and Mitigation Strategy

The framework's performance also scales effectively with query complexity, a critical factor for real-world applications where user requests can range from simple lookups to complex analytical questions.

SQL Complexity	Example Clause(s)	GPT-4 (Zero-shot) EX %	MA-SQLGrid EX %
Simple	SELECT, WHERE	~92%	~94%
Moderate	JOIN, GROUP BY	~70%	~89%
Complex	Nested Queries, HAVING, UNION,	~55%	~81%

	CTEs		
--	------	--	--

Table 6: Performance by SQL Clause Complexity on a blended dataset.

6.5 Qualitative Analysis of Complex Query Generation

To provide a more intuitive understanding of how the MA-SQLGrid framework operates, consider the following complex query from the GridDB-Maintenance dataset: *"What were the names of the technicians who performed corrective maintenance on circuit breakers that had more than three voltage spike anomalies in the last quarter?"*

A baseline model like GPT-4 (Zero-shot) might fail on this query by attempting a simple, incorrect join between Maintenance_Logs and Sensor_Readings, or by misinterpreting "last quarter." The MA-SQLGrid process unfolds as follows, as detailed in Table 7.

Agent	Input	Output / Action
NLU Analyst	Raw user query	Deconstructs the query into: Intent: Retrieve technician names. Entities: corrective maintenance, circuit breakers, > 3, voltage spike, last quarter. Ambiguity: Flags "last quarter" as needing contextual definition.
Schema Cartographer	Structured output from NLU Analyst	Links entities to schema: technicians -> Technicians.name; corrective maintenance -> Maintenance_Logs.maintenance_type, etc. Identifies join path: Technicians <->

		Maintenance_Logs <-> Assets <-> Sensor_Readings.
SQL Synthesizer	Deconstructed query + schema links & join path	Generates a candidate SQL query with a nested subquery to handle the complex conditions.
Validation Engine	Candidate SQL query	Executes the query against the database. If successful, it proceeds. If it fails (e.g., syntax error), it triggers a feedback loop for correction.
Counterfactual Reasoner	Original question and validated SQL	Generates a perturbed question: <i>"List the technicians who fixed circuit breakers that recorded over three voltage spike events in the previous quarter."</i> Verifies that the SQL generated for this new question is semantically identical to the original.

Table 7: Qualitative Walkthrough of a Complex Query

This explicit, step-by-step process is not only more robust but also inherently explainable. If a grid operator questions the result, the system can provide the entire trace of the agent interactions, showing exactly how the final query was constructed and validated. This transforms the "black box" nature of a single LLM's reasoning into an auditable and trustworthy workflow.

7. Conclusion and Future Work

7.1 Summary of Findings

This paper addressed the critical challenge of applying text-to-SQL technology to the high-stakes, complex domain of power grid database management. We identified the primary limitations of existing monolithic LLM-based approaches—namely, their lack of schema awareness, reliability, and robustness—which render them unsuitable for mission-critical industrial applications. To overcome these deficiencies, we introduced MA-SQLGrid, a novel multi-agent framework that decomposes the text-to-SQL task into a collaborative process among five specialized agents.

Our experimental results demonstrate the superiority of this paradigm. MA-SQLGrid significantly outperformed a range of competitive baselines, including state-of-the-art LLM-based models, on both standard benchmarks (Spider, WikiSQL) and a newly created, domain-specific simulated power grid database. Ablation studies confirmed that each specialized agent provides a crucial and synergistic contribution to the framework's overall performance, with the validation and schema-linking components proving most critical. Most importantly, our robustness evaluation revealed that MA-SQLGrid is remarkably resilient to adversarial perturbations in user questions and database schemas, maintaining high accuracy where monolithic models fail. This is a direct result of its unique architecture, which includes structured interaction protocols and a dedicated agent for assessing causal and counterfactual robustness.

7.2 Implications for Industrial AI Applications

The findings of this work have broader implications beyond the specific application of text-to-SQL in power grids. The core architectural principles of MA-SQLGrid—task decomposition, agent specialization, structured collaboration, iterative self-correction, and explicit robustness validation—can serve as a valuable blueprint for developing more reliable and trustworthy AI systems for a wide range of industrial applications.

In domains such as advanced manufacturing, aerospace logistics, and pharmaceutical research, where data is complex, domain-specific, and the cost of error is high, the "single intelligent agent" paradigm often proves too brittle. The multi-agent, collaborative approach proposed here offers a path toward building systems that are not only highly capable but also transparent and auditable. By externalizing the reasoning process into an observable dialogue between

specialized agents, we can move from opaque "black box" AI to systems whose decision-making processes can be understood, verified, and ultimately, trusted by human operators in critical environments.

7.3 Future Research Directions

While MA-SQLGrid establishes a strong foundation, several exciting avenues for future research remain.

- **Advanced Agent Learning and Negotiation:** The current interaction protocols are rule-based. Future work could explore more dynamic protocols where agents learn and adapt their negotiation and collaboration strategies over time using multi-agent reinforcement learning. This could lead to more efficient and context-aware problem-solving.
- **Conversational and Contextual Interaction:** The current framework is designed for single-turn question answering. Extending MA-SQLGrid to handle multi-turn, conversational interactions, where users can ask follow-up questions and clarify their intent, is a crucial next step for creating truly interactive data exploration tools.
- **Generalization to Other Structured Data Modalities:** The principles of MA-SQLGrid could be adapted to query other forms of structured data beyond relational databases. An interesting direction would be to apply a similar multi-agent architecture to the task of natural language querying of complex knowledge graphs or time-series databases.
- **Automated Agent Role Adaptation:** The agent roles in the current framework are pre-defined. Future research could investigate methods for automatically learning or adapting these roles based on the specific characteristics of a new database or domain, creating a more flexible and self-configuring system.

By pursuing these directions, the research community can continue to build upon the collaborative AI paradigm to create increasingly powerful, reliable, and trustworthy interfaces between humans and complex data systems.

References

Achiam, J., Adler, S., Agarwal, S., et al. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08776*.

Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems* 33.

Chang, Z., Gao, J., Zhang, Z., et al. (2023). Dr. Spider: A Diagnostic Evaluation Benchmark Towards Text-to-SQL Robustness. *arXiv preprint arXiv:2305.09633*.

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Chowdhery, A., Narang, S., Devlin, J., et al. (2023). PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research*, 24(248), 1-113.

Deng, X., Wang, B., Liu, B., et al. (2021). Structure-Grounded Pretraining for Text-to-SQL. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.

Dong, Y., Wang, Y., Lin, Z., et al. (2023). A Survey for In-context Learning. *arXiv preprint arXiv:2301.00234*.

Fürst, A., Glanois, C., Paris, C., et al. (2024). Robust Text-to-SQL via Database Schema-Aware Adversarial Training. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Gan, Y., Chen, X., & Purver, M. (2021). Towards Robustness of Text-to-SQL Models against Synonym Substitution. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*.

Guo, J., Zhan, Z., Gao, Y., et al. (2019). Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*.

Li, P., Zhang, T., Li, Y., et al. (2024a). Empowering Text-to-SQL with Large Language Models: A Survey. *arXiv preprint arXiv:2401.02213*.

Li, R., Allal, L. B., Zi, Y., et al. (2023c). StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*.

Liu, J., Li, C., Zhang, T., et al. (2023). A Comprehensive Survey on Text-to-SQL. *arXiv preprint arXiv:2308.07636*.

Pi, J., Zhang, Y., Lin, H., et al. (2022). Towards Robustness of Text-to-SQL Models Against Natural and Realistic Adversarial Table Perturbation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*.

Pourreza, M., & Rafiei, D. (2023). DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. *arXiv preprint arXiv:2304.11015*.

Rajkumar, N., Li, R., & Diao, Y. (2022). Text-to-SQL in the Wild: A Naturally-Occurring Dataset and Empirical Study. *arXiv preprint arXiv:2209.13524*.

Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Shen, Y., Chang, C., & Lee, D. (2023). Improving Robustness of Text-to-SQL Models via Schema-aware Adversarial Data Augmentation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Sun, Z., Zhuang, H., Li, J., et al. (2023). SQL-PaLM: Improved Large Language Model Adaptation for Text-to-SQL. *arXiv preprint arXiv:2306.00739*.

Tai, T., Liu, J., Zhang, T., et al. (2023). Exploring Chain-of-Thought Style Prompting for Text-to-SQL. *arXiv preprint arXiv:2305.10838*.

Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020). RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*.

Wang, C., Wang, J., Zhang, Z., et al. (2023). MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. *arXiv preprint arXiv:2312.11242*.

Wang, Y., Xu, K., & Reddy, C. K. (2021). Learning to Decompose and Organize Code for Text-to-SQL Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Wang, Z., Zhang, Z., Gao, J., et al. (2024). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment*, 17(4), 1132-1145.

Wei, J., Hou, L., Lampinen, A., et al. (2023). Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research*.

Xu, X., Liu, C., & Song, D. (2017). SQLNet: Generating Structured Queries from Natural Language Without Reinforcement Learning. *arXiv preprint arXiv:1711.04436*.

Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *arXiv preprint arXiv:1709.00103*.

Zhuo, H., Shen, Y., & Lee, D. (2023). Enhancing Robustness of Text-to-SQL Models via

Appendix

A.1 Example Agent Prompts

Effective prompt engineering is crucial for guiding the behavior of specialized agents within a multi-agent system. Each agent in the MA-SQLGrid framework is instantiated with a system prompt that defines its role, goal, and expected output format. The following are illustrative examples of the prompts used to initialize each agent.

A.1.1 NLU Analyst Agent Prompt

You are an expert NLU Analyst. Your task is to deconstruct a user's natural language query into a structured JSON object.

Your analysis must perform the following steps:

1. **Intent Recognition**: Identify the user's primary goal (e.g., AGGREGATE, LIST, FIND_EXTREME).
2. **Entity Extraction**: Extract all key entities from the query. For each entity, specify its type (e.g., asset_id, location, temporal_constraint, value_condition).
3. **Ambiguity Identification**: Proactively flag any terms that are ambiguous or could map to multiple schema elements (e.g., "status," "name").

User Query:

```
"{user_query}"
```

Produce a JSON object with the following structure:

```
{  
  "intent": "...",  
  "entities": [  
    {"entity_text": "...", "entity_type": "..."},  
    ...  
  ]  
}
```

```

],
"ambiguities": ["...",...]
}

```

A.1.2 Schema Cartographer Agent Prompt

You are an expert Database Schema Cartographer. Your task is to map the linguistic entities from a deconstructed user query to the provided database schema, identify the minimal relevant sub-schema required to answer the query, and determine all valid join paths.

```

**Database Schema:**
{full_database_schema}

```

```

**Deconstructed User Query:**
{nlu_analyst_output}

```

Perform the following steps:

1. **Schema Linking**: For each entity provided, map it to the most probable table and column in the full database schema.
2. **Schema Pruning**: Identify the minimal set of tables and columns necessary to answer the query. Include all tables required for joins.
3. **Join Path Identification**: Using the foreign key relationships, identify and list all valid join paths between the tables in the pruned sub-schema.

Produce a JSON object with the following structure:

```

{
  "pruned_schema": {
    "table_name_1": ["column1", "column2"],
    "table_name_2":,
    ...
  },
  "join_paths": ["table_name_1", "table_name_2", "ON table_name_1.col = table_name_2.col"],
  ...
}

```

A.1.3 SQL Synthesizer Agent Prompt

You are an expert PostgreSQL programmer. Your task is to generate a syntactically correct and semantically accurate PostgreSQL query based on the user's intent and the provided pruned schema.

****Instructions:****

- Adhere strictly to PostgreSQL syntax.
- Use ONLY the tables and columns provided in the pruned schema.
- Use the provided join paths for any necessary JOIN operations.
- If the user's intent involves complex logic, think step-by-step to construct the query.

****Pruned Schema:****

{pruned_schema}

****Join Paths:****

{join_paths}

****Deconstructed User Query:****

{nlu_analyst_output}

****Few-Shot Example:****

****User Query**:** "Show the names of technicians who worked on transformers in the 'Downtown' substation."

****SQL**:** SELECT T1.name FROM Technicians AS T1 JOIN Maintenance_Logs AS T